

Divide and Conquer

Analysis and Design of Algorithms



The Core Idea

01

Divide: Divide the given problem into sub-problems using recursion.

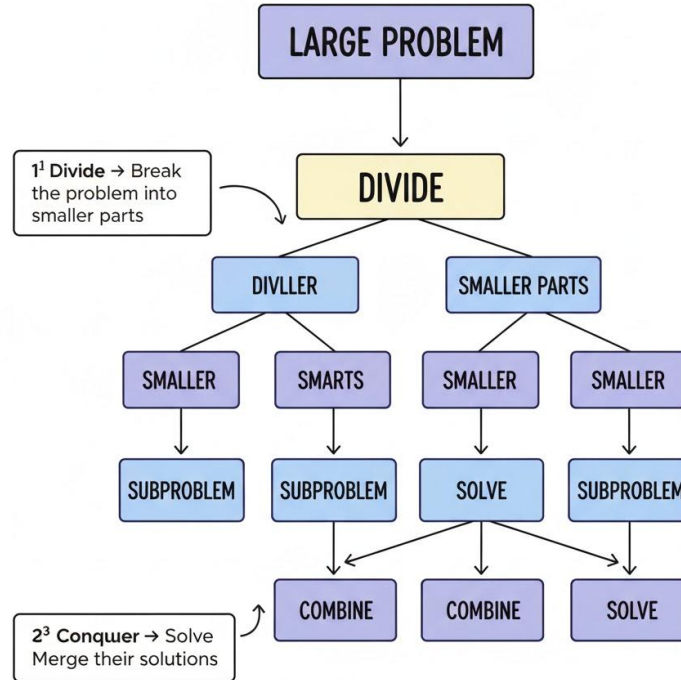
02

Conquer: Solve the smaller sub-problems recursively. If the subproblem is small enough, then solve it directly.

03

Combine: Combine the solutions of the sub-problems that are part of the recursive process to solve the actual problem.

The Core Idea



Example 1: Binary search

[3, 5, 7, 9, 11, 13, 15]

Target = 9, 15

Left side [3,5,7]

Right side [11,13,15]

Mid [9]

Left side $\rightarrow$ [3,5][7] mid -1

Right side $\rightarrow$ [11,13][15] mid+1



Example 1: Binary search

Step 1: Initialize

low = 0

high = 6

mid = $(0 + 6) // 2 = 3$

arr[mid] = 9

Compare

arr[mid] == target



Example 1: Binary search

Step 1

low = 0, high = 6

mid = 3

arr[mid] = 9

13 > 9 → search right

half → low = 4, high = 6

Step 2

mid = $(4 + 6) // 2 = 5$

arr[mid] = 13



Example 1: Binary search

Divide:

Pick the middle element (mid) of the current array segment.



Example 1: Binary search

Divide:

Pick the middle element (mid) of the current array segment.

Conquer:

If $\text{arr}[\text{mid}] == \text{target} \rightarrow \text{done}$

If $\text{target} < \text{arr}[\text{mid}] \rightarrow \text{search the left half}$

If $\text{target} > \text{arr}[\text{mid}] \rightarrow \text{search the right half}$



Example 1: Binary search

Divide:

Pick the middle element (mid) of the current array segment.

Conquer:

If $\text{arr}[\text{mid}] == \text{target} \rightarrow \text{done}$

If $\text{target} < \text{arr}[\text{mid}] \rightarrow \text{search the left half}$

If $\text{target} > \text{arr}[\text{mid}] \rightarrow \text{search the right half}$

Combine:

The “combine” is implicit

The result propagates back through the recursive calls.



Example 2: Min & Max

- Array [7,6,1,5,4,3]



Example 2: Min & Max

- **Divide** the array into two halves

Array [7,6,1,5,4,3]

[7,6,1]

[5,4,3]



Example 2: Min & Max

- Again, divide each subpart recursively into two halves until you get individual elements.

Array [7,6,1,5,4,3]

[7,6,1]
[7,6] [1]
[7] [6]
Max= 7
Min =1

[5,4,3]
[5,4] [3]
[5] [4]
max= 5
min =3



Example 2: Min & Max

- Now, combine the individual elements in a sorted manner. Here, **conquer** and **combine** steps go side by side.

[7][6]

[5][4]

[7,6][1]

[5,4][3]

[7,6,1]

[5,4,3]

Array [7,6,1,5,4,3]



Example 2: Min & Max

```
def max_min_naive(arr):  
    max_val = arr[0]  
    min_val = arr[0]  
    for i in range(1, len(arr)):  
        if arr[i] > max_val:  
            max_val = arr[i]  
        if arr[i] < min_val:  
            min_val = arr[i]  
    return max_val, min_val  
  
arr = [6, 4, 26, 14, 33, 64, 46]  
max_val, min_val = max_min_naive(arr)  
print("Maximum element is:", max_val)  
print("Minimum element is:", min_val)
```



Example 2: Min & Max Basic Method

```
def max_min(arr):  
    max_val = arr[0]  
    min_val = arr[0]  
    for i in range(1, len(arr)):  
        if arr[i] > max_val:  
            max_val = arr[i]  
        if arr[i] < min_val:  
            min_val = arr[i]  
    return max_val, min_val  
  
arr = [6, 4, 26, 14, 33, 64, 46]  
max_val, min_val = max_min(arr)  
print("Maximum element is:", max_val)  
print("Minimum element is:", min_val)
```



Example 2: Min & Max Divide & Conquer

Step 1

[6, 4, 26, 14, 33, 64, 46]

low = 0, high = 6

mid = 3

Left half: index 0–3 → [6, 4, 26, 14]

Right half: index 4–6 → [33, 64, 46]



Example 2: Min & Max Divide & Conquer

Step 2 Left Half

[6, 4, 26, 14]

low = 0, high = 3

mid = 1

Left: index 0–1 $\rightarrow$ [6, 4]

Right: index 2–3 $\rightarrow$ [26, 14]



Example 2: Min & Max Divide & Conquer

[6, 4, 26, 14]

This is a 2-element case:

$6 > 4 \rightarrow \text{max} = 6, \text{min} = 4$



Example 2: Min & Max Divide & Conquer

[6, 4, 26, 14]

This is a 2-element case:

$6 > 4 \rightarrow \text{max} = 6, \text{min} = 4$

Another 2-element case:

$26 > 14 \rightarrow \text{max} = 26, \text{min} = 14$



Example 2: Min & Max Divide & Conquer

[6, 4, 26, 14]

This is a 2-element case:

$6 > 4 \rightarrow \text{max} = 6, \text{min} = 4$

Another 2-element case:

$26 > 14 \rightarrow \text{max} = 26, \text{min} = 14$

Combine results for (0-3)

$\text{overall_max} = \text{max}(6, 26) = 26$

$\text{overall_min} = \text{min}(4, 14) = 4$



Example 2: Min & Max Divide & Conquer

Step 3 Right Half

[33, 64, 46]

low = 4, high = 6

mid = 5

Left: index 4–5 $\rightarrow$ [33, 64]

Right: index 6–6 $\rightarrow$ [46]



Example 2: Min & Max Divide & Conquer

Left [33, 64]

This is a 2-element case:

$64 > 33 \rightarrow \text{max} = 64, \text{min} = 33$

Right: [46] (index 6)

Single element $\rightarrow$ both max and min = 46



Example 2: Min & Max Divide & Conquer

Combine results for (4-6)

Left: max = 64, min = 33

Right: max = 46, min = 46

overall_max = $\max(64, 46) = 64$

overall_min = $\min(33, 46) = 33$



Example 2: Min & Max Divide & Conquer

Final Step: Combine Whole Array Results

Left half (0-3):

max = 26

min = 4

Right half (4-6):

max = 64

min = 33

Maximum element is: 64 $\text{max}(64, 26)$

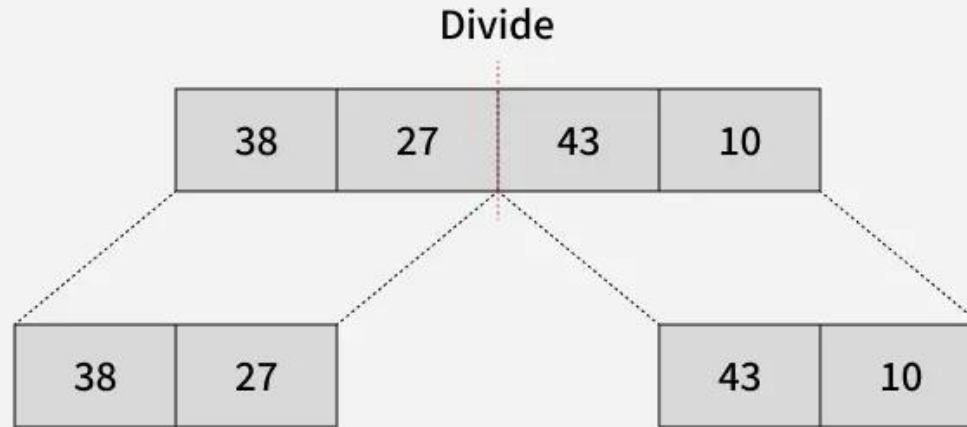
Minimum element is: 4 $\text{min}(33, 4)$



Example 3: Merge Sort

01
Step

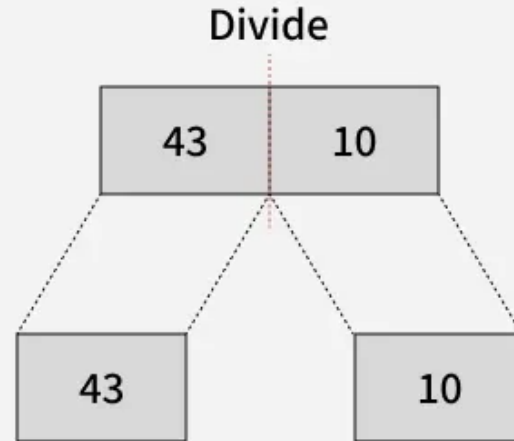
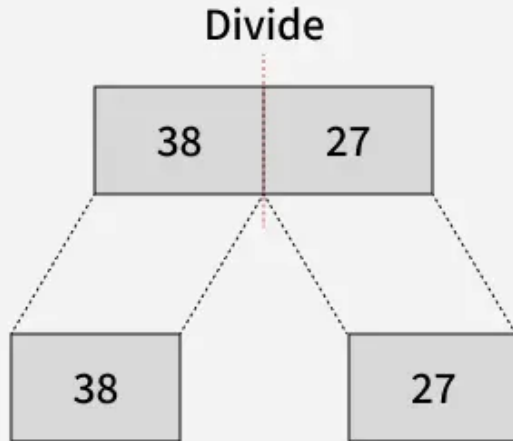
Splitting the Array into two equal halves



Example 3: Merge Sort

02
Step

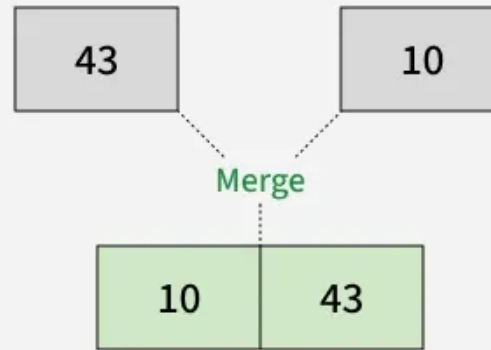
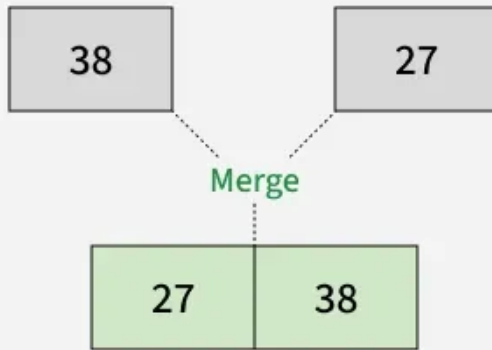
Splitting the subarrays into two halves



Example 3: Merge Sort

03
Step

Merging unit length cells into sorted subarrays



Example 3: Merge Sort

04
Step

Merging sorted subarrays into the sorted array

